Análisis y herramientas de extracción de datos: web scraping con BeautifulSoup, Selenium y Scrapy

Jesus Gonzalez

Ingeniería de Software y Datos

IU Digital

Programación para Análisis de Datos — PREICA2602B020048

Andrés Felipe Palacio

Octubre de 2026

Contenido

1. Introducción y planteamiento del problema 3
1.1 Los tres métodos como respuesta al problema 3
2. Descripción de la página y del artículo a analizar 4
2.1 La página 4
2.2 El artículo 5
3. Tema de interés de la primera práctica 6
4. Objetivos y especificación de la solución 6
4.1 Trazabilidad entre objetivos, métodos y métricas 8
5. Metodología empleada de scraping 8
5.1 Arquitectura de la solución 8
5.2 Método 1: BeautifulSoup 9
5.3 Método 2: Selenium 10
5.4 Método 3: Scrapy 11
5.5 Consideraciones éticas y legales 11
5.6 Métricas de éxito 12
6. Resultados 12
6.1 Ejecución del pipeline 12
6.2 El catálogo extraído 13
6.3 El artículo analizado 15
6.4 Precio frente a reputación 16
6.5 Comparación de métodos y concordancia 17
6.6 Validación contra las métricas de éxito 18
7. Conclusiones 18
8. Referencias 20
Apéndice A. Código fuente 21
A.1 Normalización compartida (fragmento de src/normalizers.py) 21
A.2 Extracción estática con BeautifulSoup (fragmento de src/scrapers/bs4_scraper.py) 21
A.3 Fijación del contexto de entrega con Selenium (fragmento de src/session_bootstrap.py) 22
A.4 Crawler asíncrono con Scrapy (fragmento de spiders/productos.py) 22
A.5 Ejecución 22

Análisis y herramientas de extracción de datos: web scraping con BeautifulSoup, Selenium y Scrapy

1. Introducción y planteamiento del problema

El comercio electrónico produce datos de forma continua: cada consulta a una ficha de producto devuelve un precio, una calificación, un stock y una promesa de entrega que cambian con el tiempo, el lugar desde donde se consulta y el perfil del visitante. Para un analista de datos, esa información es valiosa precisamente porque no está disponible como descarga estructurada: vive dentro del HTML de la página y solo puede aprovecharse si se extrae, se normaliza y se documenta con rigor.

El *web scraping* es la técnica que permite esa apropiación. Consiste en solicitar páginas web de forma programática y transformar su estructura en registros tabulares (Mitchell, 2024). Su aplicación exige resolver tres problemas distintos: obtener el documento, interpretar su estructura y mantener el proceso en el tiempo sin degradar el servicio del sitio de origen.

Este informe documenta la primera práctica del curso, en la que se construyó un conjunto de datos del Samsung Galaxy A57 5G publicado en Amazon y de su competencia directa. El trabajo no se limita a extraer: describe el problema, formula objetivos verificables, define métricas de éxito antes de ejecutar y evalúa el resultado contra esas métricas. La evidencia producida —código, datos, figuras y reporte de validación— es reproducible y queda disponible para las siguientes unidades del curso.

1.1 Los tres métodos como respuesta al problema

El problema no se resuelve con una sola técnica, porque la ficha de Amazon combina tres capas distintas: el HTML servido, el estado que solo existe después de interactuar con la página

y el volumen del catálogo. Por esa razón el trabajo implementa tres métodos complementarios sobre el mismo artículo y el mismo esquema de datos.

BeautifulSoup atiende la capa estática. El título, el precio, la calificación y las especificaciones del Galaxy A57 llegan en la respuesta HTTP y pueden extraerse con selectores CSS sin ejecutar JavaScript. Es el método adecuado para recorrer las dieciséis fichas del catálogo al menor costo, y su límite aparece cuando el bloque de compra se suprime por la ubicación de entrega o cuando un componente se construye en el cliente.

Selenium atiende la capa dinámica. El widget *Deliver to* exige interacción real para fijar una dirección de entrega en Estados Unidos, y el histograma de calificaciones se renderiza con JavaScript, de modo que no existe en el HTML inicial. Sin este método el precio del artículo no sería observable desde Colombia y la comparación entre métodos carecería de sentido.

Scrapy atiende la capa de escala y control. Su arquitectura de *spider* y *pipeline* permite recorrer el catálogo con concurrencia limitada, reintentos y validación por ítem; además actúa como tercera medición independiente para verificar que los otros dos métodos no introducen sesgo.

La pregunta que articula el trabajo es si tres técnicas con costos y alcances distintos producen el mismo dato cuando observan el mismo contexto comercial, y qué revela ese dato sobre el posicionamiento de precio y la reputación del Galaxy A57 frente a su competencia.

2. Descripción de la página y del artículo a analizar

2.1 La página

Amazon.com es la plataforma de comercio electrónico con mayor volumen de transacciones del mundo occidental. Su catálogo se organiza en fichas de producto identificadas

por un código **ASIN** (*Amazon Standard Identification Number*), un identificador alfanumérico de diez caracteres que hace única cada publicación.

Desde el punto de vista del scraping, la ficha de Amazon presenta cuatro características determinantes:

- **HTML generado en el servidor.** El título, el precio, la calificación y las especificaciones llegan en la respuesta HTTP, lo que permite parsearlos sin ejecutar JavaScript.
- **Personalización por ubicación.** El bloque de compra (*buybox*) depende de la dirección de entrega. Si el visitante está fuera de Estados Unidos, Amazon responde con el mensaje *"This item cannot be shipped to your selected delivery location"* y omite el precio.
- **Bloques construidos con JavaScript.** El histograma de calificaciones y otros componentes se renderizan después de la carga inicial, por lo que no aparecen en el HTML estático.
- **Defensas anti-automatización.** El sitio devuelve una página de verificación cuando detecta patrones de robot: cabeceras incompletas, ausencia de cookies o frecuencias elevadas de petición.

2.2 El artículo

El artículo seleccionado es el **Samsung Galaxy A57 5G (2026), 128 GB, color Awesome Gray**, publicado en la URL `https://www.amazon.com/dp/B0GW9TVHJB`. Se trata de un teléfono de gama media-alta con pantalla AMOLED de 6,7 pulgadas, 8 GB de RAM, triple cámara y certificación IP68. La ficha declara un precio de lista de USD 599,99 y un precio de venta de USD 499,99, con un descuento del 16,7 %.

La elección responde a tres criterios. Primero, el producto tiene volumen de reseñas suficiente (209) para analizar reputación, pero no es un artículo masivo cuya ficha cambie cada hora. Segundo, pertenece a una categoría —telefonía— donde la comparación de precios es una decisión de compra cotidiana. Tercero, su rango de precio permite contrastarlo con equipos de entrada (USD 70–150) y de gama alta (más de USD 250) dentro del mismo catálogo.

3. Tema de interés de la primera práctica

El tema que se desarrolla es la **construcción de un dataset de inteligencia de precios y reputación para telefonía móvil en marketplaces**, entendido como la base analítica que permite responder preguntas como:

- ¿En qué posición de precio se ubica un equipo dentro de su segmento?
- ¿La calificación y el volumen de reseñas guardan relación con el precio?
- ¿Qué proporción del catálogo ofrece descuentos y de qué magnitud?
- ¿Cómo se distribuye la oferta entre marcas consolidadas y marcas emergentes?

Este tema articula las cuatro unidades del curso. En la Unidad 1 se construye la fuente de datos; en la Unidad 2 se limpia y preprocesa; en la Unidad 3 se enriquece con variables derivadas —posición competitiva, índice de reputación, elasticidad— y en el cierre se modela. Sin una extracción confiable, ninguna de las etapas posteriores tendría sustento.

4. Objetivos y especificación de la solución

Objetivo general. Construir un conjunto de datos verificable del Samsung Galaxy A57 5G y de su competencia directa en Amazon, mediante la aplicación de las metodologías de scraping BeautifulSoup, Selenium y Scrapy, para analizar el posicionamiento de precio y la reputación del producto.

Objetivos específicos.

1. Implementar tres extractores independientes sobre la misma ficha y el mismo buscador, con un esquema de datos común.
2. Resolver el contexto de entrega que condiciona la publicación del precio en el sitio.
3. Normalizar los registros extraídos y verificar su calidad con métricas definidas previamente.
4. Comparar los tres métodos en cobertura de campos, concordancia y costo de ejecución.
5. Analizar el precio, la calificación, el volumen de reseñas y el *Best Sellers Rank* del artículo frente a su competencia.

Justificación. Se analiza este artículo y esta empresa porque Amazon concentra la oferta y el precio de referencia que los compradores colombianos consultan antes de decidir una compra internacional, y porque el Galaxy A57 representa el segmento donde la comparación de atributos frente al precio es más intensa. Entender cómo se posiciona un equipo en ese escaparate permite tomar decisiones de compra informadas y, desde la ingeniería, demuestra que es posible construir indicadores propios sin depender de las interfaces comerciales del sitio.

4.1 Trazabilidad entre objetivos, métodos y métricas

La solución se especifica en la tabla siguiente: cada objetivo específico se asocia con el método que lo resuelve y con la métrica que permite verificarlo. Así, la metodología descrita en la sección 5 y las métricas de la sección 5.6 no quedan como piezas sueltas, sino como la respuesta verificable a cada objetivo.

Tabla 1

Trazabilidad entre objetivos, métodos y métricas de éxito

Objetivo específico	Método que lo resuelve	Métrica de verificación
Implementar tres extractores sobre un esquema de datos común	BeautifulSoup, Selenium y Scrapy sobre el contrato `Producto`	Concordancia entre métodos ≥ 90 %
Resolver el contexto de entrega que condiciona el precio	Selenium (widget *Deliver to*) y cookies compartidas	Cobertura de campos clave ≥ 95 %, con precio observado
Normalizar los registros y verificar su calidad	Funciones puras de normalización y módulo de validación	Cobertura de campos clave ≥ 95 %
Comparar los tres métodos en cobertura y costo	Orquestador del pipeline y registro de métricas	Duración ≤ 600 s y tasa de éxito HTTP ≥ 95 %
Analizar precio, calificación y reputación del artículo	Catálogo consolidado y módulo de visualización	Catálogo ≥ 12 productos

Los cinco umbrales se fijaron antes de ejecutar el pipeline y se evalúan de forma automática en cada corrida; el detalle de su definición aparece en la sección 5.6 y el resultado obtenido, en la sección 6.6.

5. Metodología empleada de scraping

Esta sección describe la metodología con la que se construye la solución: primero la arquitectura que articula los tres métodos, después el procedimiento de cada uno y, al final, las métricas que permiten declararla exitosa.

5.1 Arquitectura de la solución

El proyecto se organizó como un paquete de Python con separación estricta de responsabilidades: configuración única, funciones puras de normalización, contrato de datos,

capa HTTP, tres scrapers, módulo de validación, módulo de visualización y un orquestador de etapas. Ningún scraper define rutas, cabeceras ni umbrales por su cuenta: todos leen de `config.py`, lo que garantiza que los tres métodos observen exactamente el mismo objetivo.

```
pipeline
├── etapa 0 · bootstrap      Selenium fija el ZIP de entrega y exporta cookies
├── etapa 1 · beautifulsoup  buscador + 16 fichas de producto
├── etapa 2 · selenium       ficha del artículo + bloques dinámicos
├── etapa 3 · scrapy         crawleo asíncrono de las 16 fichas
├── etapa 4 · consolidación  dataset único en CSV y XLSX
├── etapa 5 · validación     métricas de éxito y veredicto
└── etapa 6 · visualización  figuras del informe
```

La decisión de diseño más importante es la **sesión compartida**. Selenium abre un navegador real, interactúa con el widget *Deliver to*, fija el código postal 10001 (Nueva York) y exporta las cookies resultantes. Esas cookies se reutilizan en BeautifulSoup y en Scrapy mediante la cabecera correspondiente. De este modo, los tres métodos ven el mismo contexto comercial y sus resultados pueden compararse registro a registro. Adicionalmente se homologan el idioma y la divisa (`lc-main = en_US`, `i18n-prefs = USD`) para que los títulos y los importes sean equivalentes entre métodos.

5.2 Método 1: BeautifulSoup

Enfoque. Descarga del HTML con `requests` y extracción con selectores CSS de BeautifulSoup. Es el método más liviano: no abre navegador y procesa cada ficha en fracciones de segundo.

Selectores principales. La tabla siguiente resume el contrato de extracción; cada campo tiene selectores de respaldo porque el maquetado cambia entre variantes de ficha.

Tabla 2
Selectores CSS del método de extracción estática

Campo	Selector principal
Título	`#productTitle`
Precio	`#corePrice_feature_div span.a-price span.a-offscreen`
Precio de lista	`span.basisPrice span.a-offscreen`
Calificación	`#acrPopover`
Número de reseñas	`#acrCustomerReviewText`
Disponibilidad	`#availability span`
Especificaciones	`#productOverview_feature_div tr` y `#prodDetails tr`
Viñetas descriptivas	`#feature-bullets li`

Control de calidad. La capa HTTP compartida detecta la página anti-robot mediante marcadores textuales, reintenta con retroceso exponencial hasta tres veces y guarda cada documento en `data/raw/html/` para permitir la reproducción del análisis sin volver a golpear el sitio.

5.3 Método 2: Selenium

Enfoque. Control de un navegador Microsoft Edge en modo *headless* mediante WebDriver. A diferencia del método anterior, la extracción se hace con la API del driver — `find_element`, `get_attribute`, `execute_script`— y no con un parser de HTML.

Aportes específicos.

- **Fijación del contexto de entrega.** Es el único método capaz de resolver el widget *Deliver to*; sin este paso el precio no existe para ningún método.
- **Renderizado de bloques dinámicos.** El histograma de calificaciones (`#histogramTable`) se construye con JavaScript: en el HTML estático el

contenedor existe vacío, mientras que el navegador lo entrega completo. Esta es la evidencia concreta de la diferencia entre ambos enfoques.

- **Evidencia visual.** Se guarda una captura de pantalla de la sesión como respaldo de la ejecución.

Esperas explícitas. El scraper espera la presencia de `#productTitle`, desplaza la página por bloques para disparar la carga perezosa y aguarda hasta doce segundos la aparición del histograma antes de continuar.

5.4 Método 3: Scrapy

Enfoque. Crawler asíncrono con arquitectura de *spider*, *item* y *pipeline*. El spider no normaliza: solo selecciona texto con CSS/XPath y emite un `ProductoItem` por ficha. El pipeline convierte los textos en tipos mediante las mismas funciones puras que usan los otros dos métodos, de modo que la lógica de limpieza existe una sola vez en el proyecto.

Configuración de cortesía. `ROBOTSTXT_OBEY = True`, dos peticiones concurrentes como máximo, retardo de dos segundos con jitter del 50 %, AutoThrottle adaptativo y tres reintentos por petición. Las semillas se leen de `data/raw/seeds.json` y las cookies de `data/raw/cookies.json`, lo que desacopla el subproyecto Scrapy del resto del código.

5.5 Consideraciones éticas y legales

La extracción se limitó a 16 fichas públicas con fines académicos. No se accedió a áreas autenticadas, a datos personales ni a información de otros usuarios. Se respetó `robots.txt`, se mantuvo una frecuencia baja de peticiones y se identificó un agente de usuario real. El código y los datos quedan disponibles para auditoría del docente.

5.6 Métricas de éxito

Las métricas se definieron **antes** de ejecutar el pipeline, con el fin de que el resultado fuera evaluable y no una afirmación posterior:

Tabla 3
Métricas de éxito definidas antes de la ejecución

Métrica	Definición	Umbral
Cobertura de campos	Proporción de valores no nulos en los campos clave	≥ 95 %
Concordancia entre métodos	Acuerdo por campo entre pares de métodos, con tolerancias	≥ 90 %
Tamaño del catálogo	Registros válidos consolidados	≥ 12
Tasa de éxito HTTP	Peticiones exitosas sobre peticiones realizadas	≥ 95 %
Duración	Tiempo total del pipeline	≤ 600 s

Las tolerancias de concordancia son explícitas: 1 % en precio, 0,1 estrellas en calificación, 2 % en el contador de reseñas y 90 % de similitud textual en el título. Solo se comparan valores observados por ambos métodos; un dato ausente penaliza la cobertura, no la concordancia.

6. Resultados

6.1 Ejecución del pipeline

El pipeline completo se ejecutó en **71,9 segundos**. La tabla siguiente desglosa el comportamiento de cada método.

Tabla 4
Comportamiento de cada método de extracción

Método	Registros	Duración	Cobertura de campos	Éxito
BeautifulSoup	16	4,7 s	98,8 %	100 %
Selenium	1	23,3 s	100,0 %	100 %
Scrapy	16	43,0 s	98,8 %	100 %

La diferencia de tiempos es consistente con la naturaleza de cada técnica: BeautifulSoup opera sobre HTML ya descargado; Selenium paga el costo de iniciar un navegador y renderizar; Scrapy descarga de nuevo las dieciséis fichas respetando la espera entre peticiones.

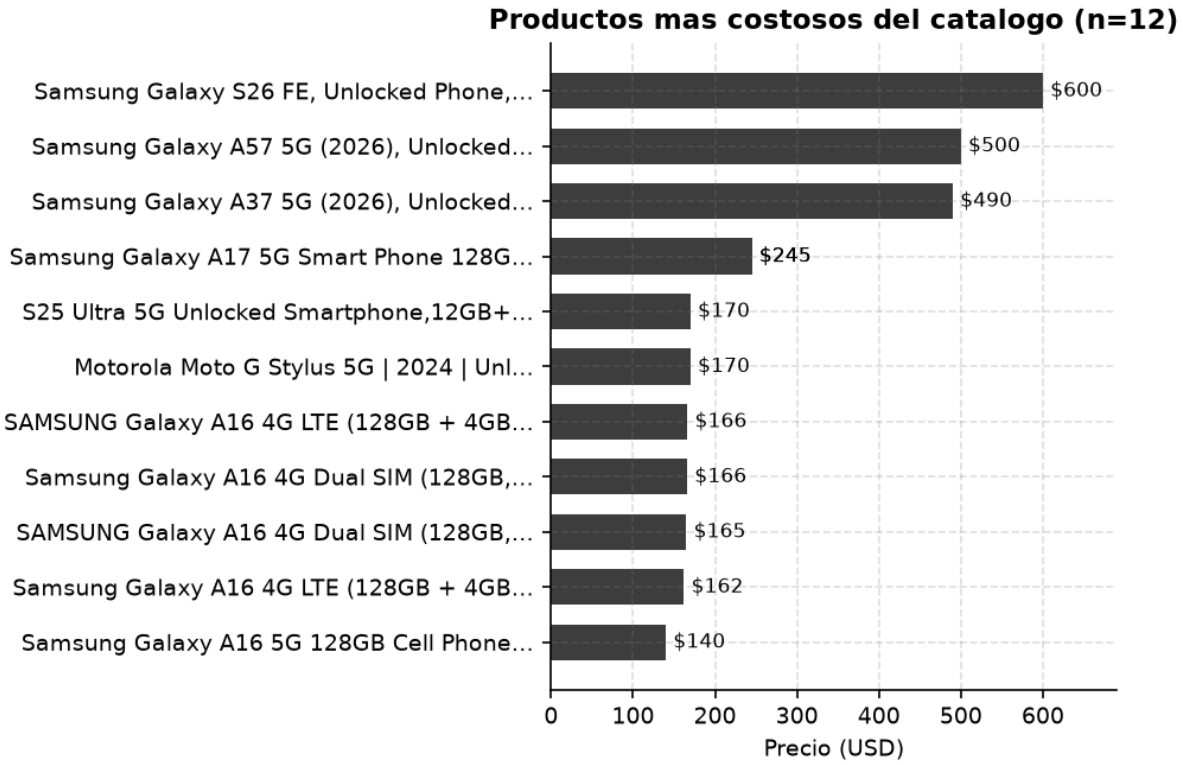
6.2 El catálogo extraído

El catálogo quedó conformado por **16 productos**, de los cuales 15 tienen precio publicado. Los precios van de **USD 69,99 a USD 599,99**, con media de USD 233,95, mediana de USD 166,29 y desviación estándar de USD 161,36: una dispersión alta, propia de un mercado que mezcla equipos de entrada con gama alta.

Por segmentos, 4 equipos se ubican por debajo de USD 150, 8 entre USD 150 y USD 250, y 3 por encima de USD 250. Samsung domina el listado con 10 de los 16 registros; el resto se reparte entre Motorola y marcas emergentes. Doce equipos están disponibles de inmediato y cuatro declaran stock limitado. Solo cuatro productos presentan descuento frente a su precio de lista, con un promedio de 12,4 %.

Figura 1

Productos más costosos del catálogo extraído (n = 12)



Nota. Elaboración propia.

6.3 El artículo analizado

Tabla 5

Ficha técnica del artículo analizado

Atributo	Valor
Título	Samsung Galaxy A57 5G (2026), 128 GB, Awesome Gray
Precio de venta	USD 499,99
Precio de lista	USD 599,99
Descuento	16,7 %
Calificación	4,4 / 5
Reseñas	209
Disponibilidad	In Stock
Entrega estimada	Sábado 24 de octubre
Sistema operativo	Android 16, One UI 8.5
RAM / almacenamiento	8 GB / 128 GB
Pantalla	6,7 pulgadas
Best Sellers Rank	6 346 en Cell Phones & Accessories; 104 en Cell Phones

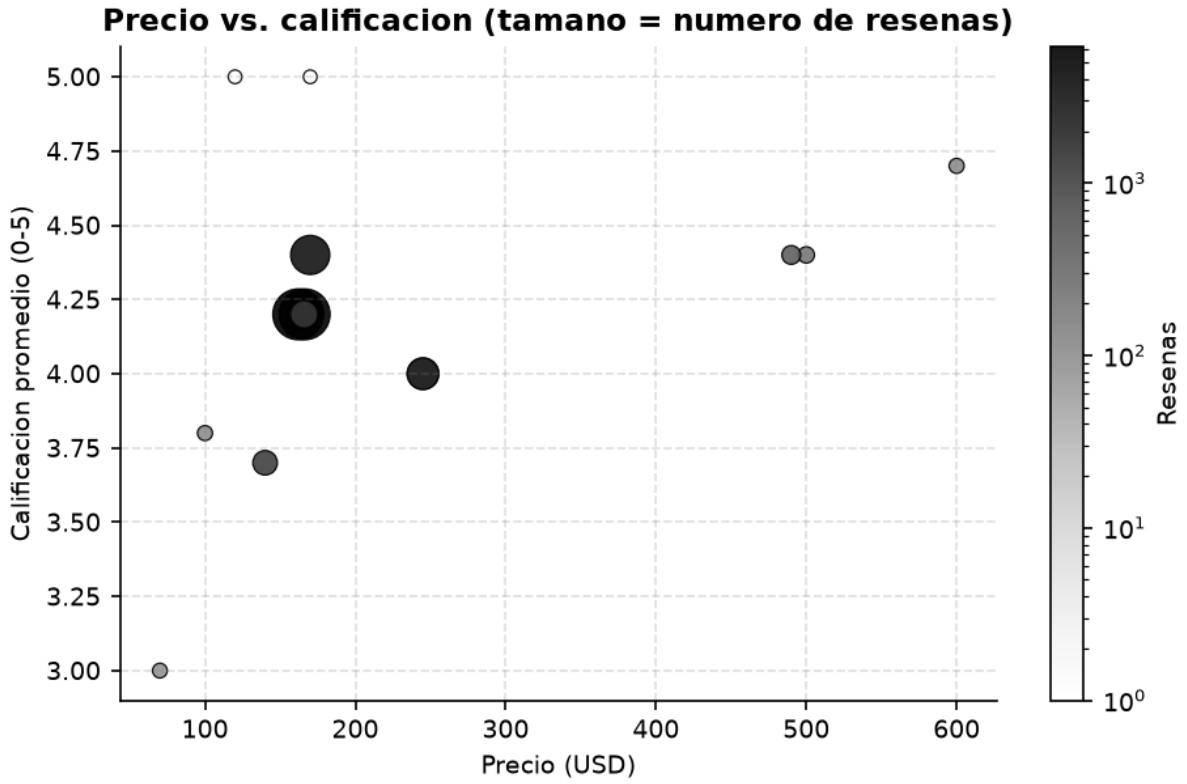
El histograma de calificaciones, obtenido únicamente con Selenium, muestra una concentración del **75 % en cinco estrellas**, 12 % en cuatro, 5 % en tres, 1 % en dos y 7 % en una. Esa distribución es la de un producto con aceptación alta pero con un núcleo minoritario de insatisfacción, un patrón habitual en telefonía de gama media.

El equipo ocupa la **segunda posición de precio** entre los quince registros con precio, solo por debajo de un modelo de USD 599,99. Su descuento de 16,7 % es el más alto del catálogo y su *Best Sellers Rank* lo sitúa en el puesto 104 de la categoría de teléfonos, un indicador de rotación saludable.

6.4 Precio frente a reputación

Figura 2

Relación entre precio y calificación; el tamaño del punto representa el número de reseñas



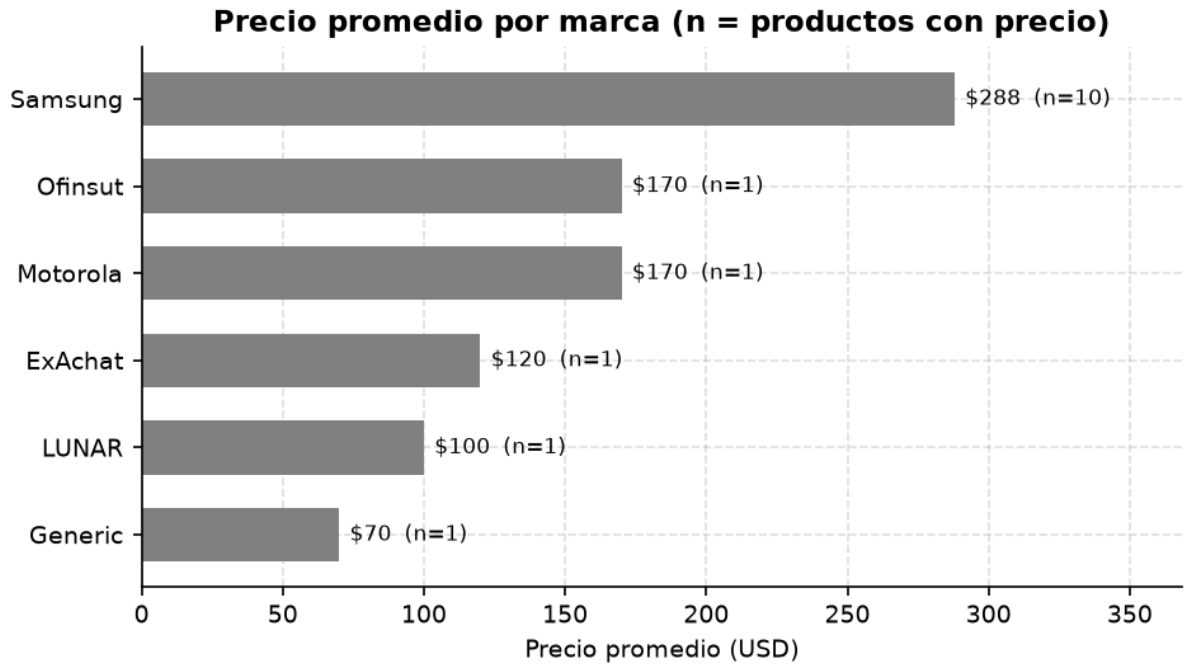
Nota. Elaboración propia.

La figura 2 muestra que la relación entre precio y calificación **no es lineal**. Los equipos con mayor volumen de reseñas se concentran entre 4,0 y 4,4 estrellas, mientras que varios productos con calificación perfecta tienen una o dos reseñas: reputación aún no consolidada. El Galaxy A57 se ubica en el cuadrante de precio alto con calificación alta y un volumen de reseñas intermedio, lo que sugiere una reputación estable pero todavía en construcción frente a los modelos más vendidos.

La calificación promedio del catálogo es de 4,19 estrellas sobre un total de 24 865 reseñas acumuladas.

Figura 3

Precio promedio y número de productos por marca

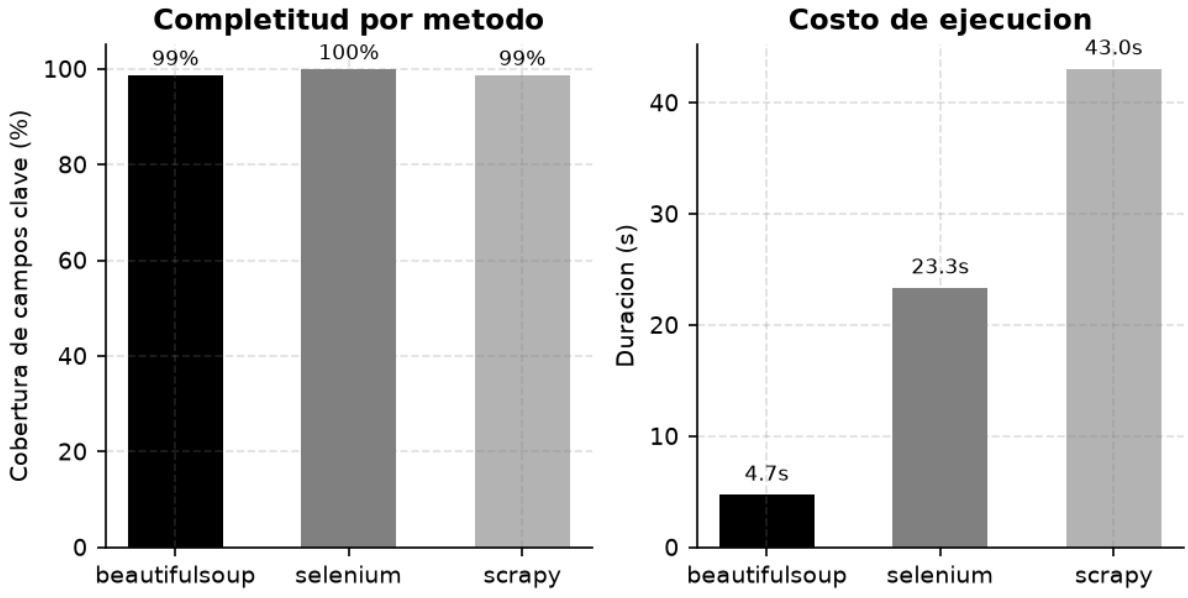


Nota. Elaboración propia.

6.5 Comparación de métodos y concordancia

Figura 4

Completitud y costo de ejecución por método



Nota. Elaboración propia.

La concordancia se calculó sobre los ASIN extraídos por más de un método. El resultado fue del **100 % en los cuatro campos comparables** —precio, calificación, reseñas y título— sobre 33 comparaciones entre pares de métodos. Este resultado valida la decisión de compartir la sesión: cuando los tres extractores observan el mismo contexto comercial, el dato coincide.

La única diferencia observada entre métodos no es de valor sino de alcance: el histograma de calificaciones solo existe en la salida de Selenium, porque se construye con JavaScript.

6.6 Validación contra las métricas de éxito

Tabla 6
Validación del pipeline contra las métricas de éxito

Métrica	Resultado	Umbral	Estado
Productos en el catálogo	16	≥ 12	Cumple
Cobertura de campos clave	98,8 %	≥ 95 %	Cumple
Concordancia entre métodos	100,0 %	≥ 90 %	Cumple
Tasa de éxito HTTP	100,0 %	≥ 95 %	Cumple
Duración del pipeline	71,9 s	≤ 600 s	Cumple

Veredicto automático: APROBADO. El reporte completo se genera en `reports/validacion.txt` y `reports/metricas.json` en cada ejecución.

7. Conclusiones

1. **Los tres métodos son viables y producen el mismo dato.** Sobre los registros comparables, la concordancia fue del 100 % en precio, calificación, reseñas y título. La diferencia entre métodos no está en la exactitud sino en el alcance y el costo.
2. **El contexto de entrega es la variable crítica del ejercicio.** Sin fijar la ubicación, Amazon suprime el bloque de compra y el precio desaparece del HTML. Selenium

resolvió ese paso y las cookies resultantes habilitaron a los otros dos métodos: la solución es un pipeline, no tres scripts aislados.

3. **Cada técnica aporta en un terreno distinto.** BeautifulSoup es el más económico para parsear HTML servido; Selenium es indispensable para el estado dinámico —fijación de ubicación e histograma de calificaciones—; Scrapy es el único que escala el catálogo con concurrencia controlada y validación por pipelines.
4. **La calidad se puede medir.** Definir métricas antes de ejecutar convirtió la entrega en un resultado verificable: 98,8 % de cobertura, 100 % de concordancia y veredicto automático aprobado.
5. **El dato tiene valor analítico.** El Galaxy A57 se posiciona en el extremo superior de su segmento (USD 499,99; segundo más costoso de quince), con 4,4 estrellas sobre 209 reseñas, el mayor descuento del catálogo y un *Best Sellers Rank* de 104 en su categoría. Su combinación de precio alto y reputación sólida pero en construcción lo convierte en un caso adecuado para el análisis de elasticidad que se abordará en las siguientes unidades.
6. **Limitaciones y trabajo futuro.** La extracción corresponde a un único corte temporal; para analizar dinámica de precios se requiere ejecutar el pipeline de forma programada y acumular series. Adicionalmente, el contador de reseñas y el *rank* cambian entre peticiones, por lo que las comparaciones deben tolerar esa variación, como ya hace el módulo de validación.

8. Referencias

Amazon. (2026). *Samsung Galaxy A57 5G (2026), unlocked Android smartphone, 128GB*. Recuperado el 19 de octubre de 2026 de https://www.amazon.com/dp/B0GW9TVHJB

Kouzis-Loukas, D. (2016). *Learning Scrapy*. Packt Publishing.

McKinney, W. (2022). *Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.

Mitchell, R. (2024). *Web scraping with Python: Collecting data from the modern web* (3rd ed.). O'Reilly Media.

Richardson, L. (2015). *Beautiful Soup documentation*. https://www.crummy.com/software/BeautifulSoup/bs4/doc/

Selenium. (2024). *Selenium WebDriver documentation*. https://www.selenium.dev/documentation/

Zelle, J. (2016). *Python programming: An introduction to computer science* (3rd ed.). Franklin, Beedle & Associates.

Apéndice A. Código fuente

El código completo se entrega en el repositorio del proyecto y en el cuaderno `Gonzalez_Jesus_Analizando_EA1.ipynb`. Se incluyen aquí los fragmentos que evidencian las tres metodologías.

A.1 Normalización compartida (fragmento de src/normalizers.py)

```
def parse_precio(texto: str | None) -> tuple[float | None, str | None]:
    """Convierte un texto de precio en (valor, moneda)."""
    limpio = limpiar_texto(texto)
    if not limpio:
        return None, None

    mayusculas = limpio.upper()
    if "COP" in mayusculas:
        moneda = "COP"
    elif "US$" in mayusculas or "$" in limpio:
        moneda = "USD"

    coincidencia = _PATRON_NUMERO.search(limpio)
    if not coincidencia:
        return None, moneda
    crudo = coincidencia.group(0)

    if moneda == "COP":
        crudo = crudo.replace(".", "").replace(",", ".")
    elif "," in crudo and "." in crudo:
        crudo = crudo.replace(",", "")
    return round(float(crudo), 2), moneda
```

A.2 Extracción estática con BeautifulSoup (fragmento de src/scrapers/bs4_scraper.py)

```
def _extraer_precio(soup: BeautifulSoup) -> dict[str, Any]:
    precio, moneda = parse_precio(_primer_texto(soup, SELECTORES["precio"]))
    precio_lista, _ = parse_precio(_primer_texto(soup, SELECTORES["precio_lista"]))
    return {
        "precio": precio,
        "moneda": moneda,
        "precio_lista": precio_lista,
        "descuento_pct": calcular_descuento(precio, precio_lista),
    }

def parse_producto(html: str, asin: str, metodo: str = METODO) -> Producto:
    soup = BeautifulSoup(html, "lxml")
    overview = _tabla_clave_valor(soup, SELECTORES["overview"][0])
    detalles = _tabla_clave_valor(soup, SELECTORES["detalles"][0])
    campos = {
        **_extraer_identidad(soup, asin, overview),
        **_extraer_precio(soup),
        **_extraer_reputacion(soup),
        **_extraer_logistica(soup),
        **_extraer_especificaciones(soup, overview, detalles),
        **_extraer_rankings(soup, detalles),
        "metodo": metodo,
    }
    return Producto(**campos)
```

A.3 Fijación del contexto de entrega con Selenium (fragmento de src/session_bootstrap.py)

```
def fijar_ubicacion(driver, zip_code: str = DELIVERY_ZIP) -> str:
    driver.get(_PAGINA_DE_APOYO)
    WebDriverWait(driver, 25).until(
        EC.presence_of_element_located((By.ID, "nav-global-location-popover-link"))
    )
    driver.find_element(By.ID, "nav-global-location-popover-link").click()
    campo = WebDriverWait(driver, 15).until(
        EC.element_to_be_clickable((By.ID, "GLUXZipUpdateInput"))
    )
    campo.clear()
    campo.send_keys(zip_code)
    driver.find_element(By.ID, "GLUXZipUpdate").click()
    time.sleep(4)
    return driver.find_element(By.ID, "glow-ingress-line2").text.strip()
```

A.4 Crawler asíncrono con Scrapy (fragmento de spiders/productos.py)

```
async def start(self) -> AsyncIterator[scrapy.Request]:
    """Emite una peticion por ASIN con las cookies de sesion."""
    for asin in self._asins():
        yield scrapy.Request(
            url=TARGET.product_url(asin),
            callback=self.parse_producto,
            cb_kwargs={"asin": asin},
            cookies=self.cookies,
            dont_filter=True,
        )

def parse_producto(self, response: Response, asin: str):
    if response.status != 200 or not response.css("#productTitle"):
        return
    yield ProductoItem(
        asin=asin,
        url=response.url,
        titulo=self._texto(response, "#productTitle"),
        precio_texto=response.css("#corePrice_feature_div span.a-price span.a-
offscreen::text").get(),
        calificacion_texto=response.css("#acrPopover::attr(title)").get(),
        resenas_texto=response.css("#acrCustomerReviewText::text").get(),
    )
```

A.5 Ejecución

```
# Pipeline completo: bootstrap + tres métodos + validación + figuras
python -m src.pipeline

# Pruebas unitarias de las funciones de normalización
python -m unittest discover -s tests -v
```